

Code:

```
#include <stdio.h> #include <string.h>

#define MAXN 505 #define MAXM 2005

int head[MAXN], to[MAXM], cap[MAXM], nxt[MAXM], edge_cnt = 1; int level[MAXN],
q[MAXN], ptr[MAXN];

void add_edge(int u, int v, int c) { to[++edge_cnt] = v; cap[edge_cnt] = c; nxt[edge_cnt] =
head[u]; head[u] = edge_cnt; to[++edge_cnt] = u; cap[edge_cnt] = 0; nxt[edge_cnt] =
head[v]; head[v] = edge_cnt; }

int bfs(int n, int s, int t) { memset(level, -1, sizeof(int) * (n + 1)); int front = 0, back = 0;
q[back++] = s; level[s] = 0;

while (front < back) {
    int u = q[front++];
    for (int e = head[u]; e; e = nxt[e]) {
        int v = to[e];
        if (cap[e] > 0 && level[v] == -1) {
            level[v] = level[u] + 1;
            q[back++] = v;
        }
    }
}
return level[t] != -1;
}

int dfs_flow(int u, int t, int pushed) { if (pushed == 0 || u == t) return pushed; for (int *cid =
&ptr[u]; *cid; *cid = nxt[*cid]) { int e = *cid; int v = to[e]; if (level[u] + 1 != level[v] || cap[e]
== 0) continue; int tr = dfs_flow(v, t, pushed < cap[e] ? pushed : cap[e]); if (tr == 0)
continue; cap[e] -= tr; cap[e ^ 1] += tr; return tr; } return 0; }

int dinic(int n, int s, int t) { int flow = 0; while (bfs(n, s, t)) { for (int i = 1; i <= n; ++i) ptr[i] =
head[i]; while (1) { int pushed = dfs_flow(s, t, 1e9); if (!pushed) break; flow += pushed; } }
return flow; }

int path[MAXN], path_len;
```

```

void get_route(int u, int t) { path[path_len++] = u; if (u == t) return; for (int e = head[u]; e; e = nxt[e]) { if (e % 2 == 0 && cap[e] == 0) { cap[e] = -1; // Mark used get_route(to[e], t); return; }}}

int main() { int n, m; if (scanf("%d %d", &n, &m) != 2) return 0;

for (int i = 0; i < m; ++i) {
    int a, b;
    scanf("%d %d", &a, &b);
    add_edge(a, b, 1);
}

int max_days = dinic(n, 1, n);
printf("%d\n", max_days);

for (int i = 0; i < max_days; ++i) {
    path_len = 0;
    get_route(1, n);
    printf("%d\n", path_len);
    for (int j = 0; j < path_len; ++j) {
        printf("%d%c", path[j], (j == path_len - 1) ? '\n' : ' ');
    }
}

return 0;

}

```

Input:

```

6 7
1 2
1 3
2 6
3 4
3 5
4 6
5 6

```

Output:

2

4

1 3 5 6

3

1 2 6